CHRONOCHEF: OBJECT-ORIENTED PROGRAMMING (OOP) IMPLEMENTATION & ARCHITECTURAL MAPPING GUIDE

KTU PBCST304: OBJECT ORIENTED PROGRAMMING — B.TECH AI & ML

Course & Syllabus:	KTU Course PBCST304: Object Oriented Programming (2024 Scheme)
Institution & Department:	Dept. of Artificial Intelligence & Machine Learning, Marian Engineering College
Project Team:	Harikrishnan P (Lead - MEC25AIM029), Athira L S (MEC25AIM019), Abhichandhana C R (MEC25AIM003), Aparna G (MEC25AIM016)
Supervision & Evaluation:	Guide: Ms. Aswathy T K (Asst. Prof.) | HOD: Prof. Balu John

This technical document provides a rigorous, code-level architectural mapping of all Object-Oriented Programming concepts implemented in the ChronoChef system. It details the exact source files, classes, methods, line numbers, and execution rationales, serving as an authoritative evaluation reference for academic viva voce and syllabus compliance verification.

Master OOP Concept Implementation Matrix

#	OOP Concept	Core Java Mechanism	File & Class Location	System Role & Academic Purpose
1	**Classes & Objects**	Class declaration, instantiation via 'new', heap memory allocation	model/FoodItem.java model/DistributionRecord.java	Models real-world relief pantry entities with encapsulated state and behavior.
2	**Encapsulation & Data Hiding**	private instance fields, public getters, validating setters	model/FoodItem.java (L13-24, L60-120) model/DistributionRecord.java	Protects state integrity; prevents non-positive quantities and invalid base pricing.
3	**Abstract Classes & Methods**	abstract class FoodItem, abstract String getStorageInstructions()	model/FoodItem.java (L11, L139) Inherited by subclasses	Establishes template entity contract requiring specific food subclasses to define storage.
4	**Hierarchical Inheritance**	extends keyword; specialized domain entity subclasses	model/PerishableItem.java (L7) model/PackagedItem.java (L7)	Reuses base properties while adding cold-chain temps and packaging seal validations.
5	**Dynamic Polymorphism (Dispatch)**	@Override annotation; dynamic method overriding at runtime	model/PerishableItem.java (L29-32) model/PackagedItem.java (L27-30)	JVM determines whether to display refrigeration or dry storage directives at runtime.
6	**Static Polymorphism**	Overloaded method	model/FoodItem.java (L25-58)	Permits flexible object initialization

	(Overloading)	signatures and overloaded constructors	model/InventoryOperations.java (L18-24)	and multi-criteria inventory lookup (ID/Name/Cat).
7	**Interfaces & Realization**	interface keyword, implements keyword, contract realization	model/InventoryOperations.java (L10) database/FoodBankDAO.java (L15)	Decouples business logic from persistence; guarantees strict contract for CRUD.
8	**Custom Exception Handling**	extends Exception, checked exceptions, throw & throws	exception/ExpiredItemDistributionException.java exception/InvalidQuantityException.java	Enforces food safety guardrail; strictly prevents checkout of expired food items.
9	**Exception Handling (Robustness)**	try-catch-finally blocks, multi-catch, safe resource closure	database/DatabaseManager.java ui/DistributionPanel.java (L180-230)	Handles SQL failures gracefully and releases database Statement/Connection locks safely.
10	**Singleton Design Pattern**	private static instance, private constructor, synchronized getInstance()	database/DatabaseManager.java (L14-25)	Guarantees exactly one thread-safe SQLite connection across the entire application.
11	**MVC Architecture**	Strict package decoupling across model, view, controller/service	model/, ui/, service/, database/ packages	Separates GUI rendering from computational decay math and SQLite persistence.
12	**Generics & Collections**	Vector<FoodItem>, Vector<DistributionRecord>	model/InventoryOperations.java ui/InventoryPanel.java (L140-190)	Provides compile-time type safety and synchronized backing data for Swing tables.
13	**JDBC Persistence**	Connection, PreparedStatement (parameterized), ResultSet	database/DatabaseManager.java database/FoodBankDAO.java	Protects against SQL Injection while executing ACID compliant storage operations.
14	**Event-Driven GUI Model**	Java Swing JFrame, JPanel, CardLayout, ActionListener	ui/MainFrame.java (L18-120) ui/InputPanel.java	Provides authentic desktop UI; handles user events asynchronously via EDT.
15	**Algorithmic In-Memory Sorting**	Custom Bubble Sort algorithm ordering items by PDI score	service/PriorityPricingService.java (L110-145)	Sorts pantry items so that urgently expiring food appears first for rapid dispatch.

1. Classes, Objects, State & Behavior

A class is a blueprint from which individual objects are created. In ChronoChef, classes encapsulate the real-world entities of a community relief operation. Objects maintain instance state in heap memory and expose behavioral methods to operate on that state.

- **Implementation Location:** model/FoodItem.java (Lines 11-170) and model/DistributionRecord.java (Lines 11-154).
- **Object Instantiation:** Main.java (Line 42) and database/DataSeeder.java (Lines 35-75) using new FoodItem subclass instances.

```
// Example from database/DataSeeder.java: Object instantiation on the JVM Heap
FoodItem milk = new PerishableItem(
    "FD-1001", "Organic Whole Milk (1L)", "Dairy",
    25.0, "Litres", "2026-10-20", "2026-10-29", 45.0, 4.0
);
FoodItem rice = new PackagedItem(
    "FD-1005", "Basmati Rice (5kg Bag)", "Grains",
    40.0, "Bags", "2026-10-01", "2026-11-20", 250.0, "Triple-Layer Foil Seal"
);
```

> 📌 **Academic Context:** Demonstrates Module 1 syllabus requirements: class structure, object creation via 'new', constructor execution, heap reference management, and separation of instance state from class definitions.

2. Encapsulation & Data Hiding

Encapsulation is the mechanism that binds together code and the data it manipulates, keeping both safe from outside interference and misuse. In ChronoChef, all internal state variables are declared strictly private, accessible solely through public getters and validating setters.

- **Implementation Location:** model/FoodItem.java (Lines 13-24, 60-120).
- **Integrity Verification:** setQuantity(double) enforces non-negative inventory levels; setBasePrice(double) prevents zero or negative retail valuation.

```
// Encapsulation with defensive setter validation in model/FoodItem.java
private String itemId;
private String name;
private double quantity;
private double basePrice;

public double getQuantity() {
    return this.quantity;
}

public void setQuantity(double quantity) {
    if (quantity < 0) {
        throw new IllegalArgumentException("Inventory quantity cannot be negative: " + quantity);
    }
    this.quantity = quantity;
}
```

3. Abstract Classes & Abstract Methods

An abstract class cannot be instantiated directly and serves as a formal superclass template. ChronoChef defines FoodItem as an abstract class containing common inventory metadata, while declaring getStorageInstructions() as an abstract method to compel all concrete food categories to provide specialized environmental preservation guidelines.

- **Implementation Location:** model/FoodItem.java (Line 11, Line 139).
- **Abstract Signature:** public abstract String getStorageInstructions();

```
// Abstract Class Template in model/FoodItem.java
public abstract class FoodItem {
    protected String itemId;
    protected String name;
    // ... Common state fields ...

    /**
     * Abstract method enforcing specialized preservation directives.
     * Must be implemented by all derived concrete subclasses.
     */
    public abstract String getStorageInstructions();
}
```

4. Hierarchical Inheritance

Inheritance is the mechanism by which one class acquires the properties and behaviors of another class. ChronoChef implements hierarchical inheritance where multiple specialized subclasses extend the common FoodItem abstract parent. Additionally, custom system exceptions extend the standard Java Exception class.

- **Cold-Chain Subclass:** model/PerishableItem.java extends FoodItem (adding requiredTempCelsius).
- **Dry Goods Subclass:** model/PackagedItem.java extends FoodItem (adding packagingType).
- **Exception Subclasses:** exception/ExpiredItemDistributionException.java extends Exception.

```
// Hierarchical Subclass 1: Cold-Chain Food Items (model/PerishableItem.java)
public class PerishableItem extends FoodItem {
    private double requiredTempCelsius;

    public PerishableItem(String itemId, String name, String category, double quantity,
                          String unit, String dateReceived, String expiryDate,
                          double basePrice, double requiredTempCelsius) {
        super(itemId, name, category, quantity, unit, dateReceived, expiryDate, basePrice);
        this.requiredTempCelsius = requiredTempCelsius;
    }
}

// Hierarchical Subclass 2: Ambient Packaged Goods (model/PackagedItem.java)
public class PackagedItem extends FoodItem {
    private String packagingType;

    public PackagedItem(String itemId, String name, String category, double quantity,
                        String unit, String dateReceived, String expiryDate,
                        double basePrice, String packagingType) {
        super(itemId, name, category, quantity, unit, dateReceived, expiryDate, basePrice);
        this.packagingType = packagingType;
    }
}
```

5. Dynamic Polymorphism (Runtime Method Dispatch)

Dynamic Polymorphism is the process by which a call to an overridden method is resolved at runtime rather than compile-time. In ChronoChef, when iterating over a heterogeneous collection of FoodItem references, calling item.getStorageInstructions() invokes the specialized subclass implementation dynamically based on the actual object instance present on the heap.

- **PerishableItem Override:** Returns cold-chain temperature directives (e.g., 'Keep Refrigerated at 4.0°C').
- **PackagedItem Override:** Returns dry storage instructions and packaging seal integrity details.

```
// Dynamic Method Dispatch at runtime
@Override
public String getStorageInstructions() {
    return "Keep Refrigerated at " + this.requiredTempCelsius + "°C (Cold-Chain)";
}

// In ui/InventoryPanel.java: Polymorphic execution across mixed items
for (FoodItem item : inventoryVector) {
    // JVM dynamically resolves getStorageInstructions() based on actual object type
    String storageInfo = item.getStorageInstructions();
    tableModel.addRow(new Object[]{ item.getName(), item.getCategory(), storageInfo });
}
```

6. Static Polymorphism (Method & Constructor Overloading)

Static (compile-time) polymorphism occurs when multiple methods or constructors in the same class share the same identifier but possess distinct parameter lists (different arity or types). ChronoChef implements constructor overloading for entity instantiation and method overloading for flexible inventory querying.

- **Constructor Overloading:** model/FoodItem.java provides full parameterized constructor and minimal fallback constructor.
- **Method Overloading:** model/InventoryOperations.java and database/FoodBankDAO.java declare searchItems(keyword), searchItems(category, onlyActive), and getItemById(itemId).

```
// Overloaded Query Methods in model/InventoryOperations.java
public Vector<FoodItem> searchItems(String keyword);
public Vector<FoodItem> searchItems(String category, boolean onlyActive);
public FoodItem getItemById(String itemId);
```

7. Interfaces & Contractual Realization

An interface is a completely abstract type used to specify a contractual behavior that classes must implement. ChronoChef defines InventoryOperations to enforce a strict boundary between user interface controllers and the underlying database access layer, achieving 100% loose coupling.

- **Interface Definition:** model/InventoryOperations.java (Lines 10-44).
- **Interface Realization:** database/FoodBankDAO.java implements InventoryOperations (Lines 15-381).

```
// Contractual Interface in model/InventoryOperations.java
public interface InventoryOperations {
    public boolean addItem(FoodItem item);
    public boolean updateStock(String itemId, double newQuantity);
    public boolean deleteItem(String itemId);
    public Vector<FoodItem> getAllItems();
    public FoodItem getItemById(String itemId);
    public boolean recordDistribution(DistributionRecord record)
        throws ExpiredItemDistributionException, InvalidQuantityException;
}

// Realization in database/FoodBankDAO.java
public class FoodBankDAO implements InventoryOperations {
    @Override
    public boolean addItem(FoodItem item) { /* JDBC PreparedStatement Insert */ }

    @Override
    public boolean recordDistribution(DistributionRecord record)
        throws ExpiredItemDistributionException, InvalidQuantityException {
        // Enforces safety guardrails prior to executing SQL transaction
    }
}
```

8. Custom Exception Handling & Safety Guardrails

Custom exceptions extend the standard Exception hierarchy to represent domain-specific fault states. ChronoChef creates custom checked exceptions to enforce strict public health safety rules and inventory volume invariants.

- **Expired Food Barrier:** exception/ExpiredItemDistributionException.java ensures no volunteer can distribute expired goods.
- **Quantity Barrier:** exception/InvalidQuantityException.java blocks zero, negative, or overdraft checkout volumes.

```
// Custom Checked Exception enforcing Public Health Safety
package exception;

public class ExpiredItemDistributionException extends Exception {
    private String itemId;
    private int daysRemaining;

    public ExpiredItemDistributionException(String itemId, String itemName, int daysRemaining) {
        super("CRITICAL SAFETY VIOLATION: Cannot distribute expired food item [" +
              itemId + " - " + itemName + "]! Expiry was " + Math.abs(daysRemaining) + " days ago.");
        this.itemId = itemId;
        this.daysRemaining = daysRemaining;
    }

    public int getDaysRemaining() { return daysRemaining; }
}
```

9. Robust Exception Handling (Try-Catch-Finally Architecture)

ChronoChef uses structured try-catch-finally blocks to intercept checked exceptions, display user-friendly modal alerts in Java Swing, and ensure database Statement and Connection resources are closed deterministically in all execution paths.

```
// Safe Resource Management & Safety Interception in ui/DistributionPanel.java
try {
    dao.recordDistribution(record);
    JOptionPane.showMessageDialog(this, "Distribution Completed Successfully!", "Success",
JOptionPane.INFORMATION_MESSAGE);
} catch (ExpiredItemDistributionException ex) {
    JOptionPane.showMessageDialog(this, ex.getMessage(), "Food Safety Violation",
JOptionPane.ERROR_MESSAGE);
} catch (InvalidQuantityException ex) {
    JOptionPane.showMessageDialog(this, ex.getMessage(), "Input Quantity Error",
JOptionPane.WARNING_MESSAGE);
} catch (SQLException ex) {
    JOptionPane.showMessageDialog(this, "Database Transaction Failed: " + ex.getMessage(), "SQL Error",
JOptionPane.ERROR_MESSAGE);
} finally {
    refreshTable(); // Always update UI state regardless of success or failure
}
```

10. Singleton Design Pattern

The Singleton pattern ensures that a class has only one instance while providing a global access point to that instance. In ChronoChef, DatabaseManager implements the Singleton pattern with synchronized initialization to prevent multiple threads from establishing concurrent, conflicting SQLite file locks.

- **Implementation Location:** database/DatabaseManager.java (Lines 14-25).
- **Pattern Elements:** 1) private static instance; 2) private constructor; 3) public static synchronized getInstance().

```
// Thread-Safe Singleton Implementation in database/DatabaseManager.java
public class DatabaseManager {
    private static DatabaseManager instance;
    private Connection connection;

    // 1. Private constructor blocks direct instantiation from external classes
    private DatabaseManager() {
        initConnection();
        createTables();
    }

    // 2. Synchronized global access point
    public static synchronized DatabaseManager getInstance() {
        if (instance == null) {
            instance = new DatabaseManager();
        }
        return instance;
    }

    public Connection getConnection() {
        return this.connection;
    }
}
```

11. Model-View-Controller (MVC) Architectural Pattern

ChronoChef strictly decouples data representation, presentation, and algorithmic processing across four clean packages:

- **Model (model/):** Encapsulates food items, relief distribution records, and contractual interfaces.
- **View (ui/):** Java Swing graphical panels (Input, Inventory, Management, Distribution, Analytics) organized under MainFrame.
- **Controller / Service (service/, database/):** Computes PDI decay scores, applies tiered subsidies, and persists state via FoodBankDAO.

12. Java Collections & Generics

Generics introduce compile-time type safety, eliminating the necessity for explicit type casting. ChronoChef utilizes Vector<FoodItem> and Vector<DistributionRecord> as synchronized collections that interface directly with Swing DefaultTableModel.

```
// Type-Safe Collection Declaration in model/InventoryOperations.java
Vector<FoodItem> getAllItems();
Vector<DistributionRecord> getAllDistributions();

// Thread-safe table iteration in ui/AnalyticsPanel.java
Vector<DistributionRecord> records = dao.getAllDistributions();
for (DistributionRecord rec : records) {
    totalKgDistributed += rec.getDistributedQty();
    totalSubsidySaved += (rec.getBasePrice() - rec.getEffectivePrice()) * rec.getDistributedQty();
```

```
}
```

13. JDBC Persistence & SQL Injection Defense

Java Database Connectivity (JDBC) standardizes relational database interactions. In database/FoodBankDAO.java, all queries are executed via parameterized PreparedStatement objects, completely eliminating SQL Injection vulnerabilities while ensuring atomic database commits.

```
// Parameterized PreparedStatement in database/FoodBankDAO.java
String sql = "INSERT INTO food_items (item_id, name, category, item_type, quantity, unit, " +
             "date_received, expiry_date, base_price, storage_info) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?,
?)";

try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
    pstmt.setString(1, item.getItemId());
    pstmt.setString(2, item.getName());
    pstmt.setString(3, item.getCategory());
    pstmt.setString(4, item instanceof PerishableItem ? "PERISHABLE" : "PACKAGED");
    pstmt.setDouble(5, item.getQuantity());
    pstmt.setString(6, item.getUnit());
    pstmt.setString(7, item.getDateReceived());
    pstmt.setString(8, item.getExpiryDate());
    pstmt.setDouble(9, item.getBasePrice());
    pstmt.setString(10, item.getStorageInstructions());
    pstmt.executeUpdate();
}
```

14. Event-Driven GUI Programming (Java Swing & AWT)

ChronoChef is constructed with Java Swing and AWT layout managers, coordinating views through CardLayout and the Delegation Event Model. Thread safety is guaranteed by launching the UI on the Event Dispatch Thread (EDT).

- **EDT Lifecycle:** SwingUtilities.invokeLater(() -> new MainFrame().setVisible(true)) in Main.java.
- **Delegation Listeners:** ActionListener attached to donation intake, category filters, and checkout buttons.
- **CardLayout Navigation:** MainFrame switches between 5 modular view panels seamlessly without recreating windows.

15. Algorithmic In-Memory Sorting (Custom Bubble Sort)

To fulfill the First-Expired, First-Out (FEFO) food banking principle, ChronoChef implements an in-memory Bubble Sort algorithm in service/PriorityPricingService.java. The algorithm sorts Vector<FoodItem> in descending order of the Priority Distribution Index (PDI), ensuring critical near-expiry batches appear at the top of the volunteer screen.

```
// In-Memory Bubble Sort prioritizing food items by PDI score (service/PriorityPricingService.java)
public static void sortByPriorityDescending(Vector<FoodItem> items) {
    if (items == null || items.size() <= 1) return;
    int n = items.size();
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            int pdi1 = calculatePDI(getDaysRemaining(items.get(j).getExpiryDate()));
            int pdi2 = calculatePDI(getDaysRemaining(items.get(j + 1).getExpiryDate()));
            // Swap if preceding item has lower priority score
            if (pdi1 < pdi2) {
                FoodItem temp = items.get(j);
                items.set(j, items.get(j + 1));
                items.set(j + 1, temp);
            }
        }
    }
}
```

16. Viva Voce Examination Q&A Cheat Sheet

Comprehensive answers to the most probable OOP architecture questions asked during university evaluation:

Q1: Why did you declare FoodItem as an abstract class instead of an interface?

Answer: FoodItem represents a foundational entity with substantial common state (itemId, name, quantity, basePrice) and concrete setter validation methods that are shared across all food categories. An interface cannot hold mutable instance state. Making FoodItem an abstract class allows code reuse for common attributes while enforcing getStorageInstructions() as an abstract method for specialized subclasses.

Q2: Where is Dynamic Method Dispatch demonstrated in your project?

Answer: It occurs when calling item.getStorageInstructions() on a generic FoodItem reference. If the object on the heap is a PerishableItem, the JVM dynamically invokes PerishableItem.getStorageInstructions() to return refrigeration temperatures. If it is a PackagedItem, it dynamically executes PackagedItem.getStorageInstructions() to return seal integrity data.

Q3: Why is DatabaseManager implemented as a Singleton?

Answer: SQLite operates in file-locking mode. Creating multiple database manager instances across different UI panels would lead to concurrent connection attempts, database lock exceptions, and connection leaks. The Singleton pattern guarantees exactly one thread-safe connection instance globally.

Q4: How do your custom exceptions contribute to software robustness?

Answer: ExpiredItemDistributionException and InvalidQuantityException are checked exceptions. By declaring them on the DAO distribution method signature, the compiler forces any calling UI controller to wrap checkouts in a try-catch block, guaranteeing that expired food can never be checked out under any operational circumstance.

Q5: What is the purpose of the InventoryOperations interface?

Answer: It enforces the Dependency Inversion Principle. The UI views (DistributionPanel, InputPanel) program against the InventoryOperations interface contract rather than the concrete FoodBankDAO implementation. This allows swapping the SQLite storage engine with PostgreSQL or cloud mock engines without modifying any UI code.